

Session 01 - Artificial Intelligence in Interactive Systems

Introduction to Machine Learning

Machine Learning is a subset of AI that enables systems to learn from data without explicit programming for a concrete problem.

A really good quick introduction!

If you want to get a quick understanding of the basic topics in this 'script' and also a little headstart you might want to read this short *interactive* blog article, as it is a great starting point: [What is a Convolutional Neural Network?](#).

And Grant Sanderson aka 3blue1brown also covers a lot of ground fast in his: [Playlist about Neural Networks](#)

Ubiquitous Computing - Mark Weiser

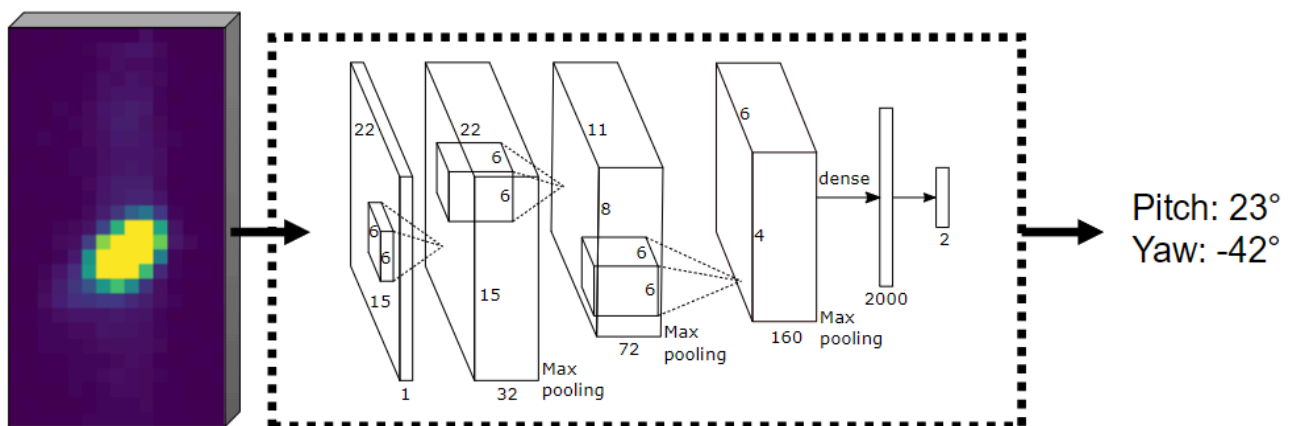
"The most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it" "...Hundreds of computers in a room could seem intimidating at first, [...] these hundreds of computers will come to be invisible to common awareness. People will simply use them unconsciously to accomplish everyday tasks."

What is meant by Ubiquitous Computing?

It refers to a paradigm where computing is everywhere around the user at any time. Instead of users actively seeking out computers, computing devices are seamlessly integrated into the environment, becoming invisible and assisting users in their daily tasks unconsciously.

Understanding the context of the Human

Enhancing Touch Input through Machine Learning: Instead of only interpreting the 2D touch coordinate passed through the touch-screen to the kernel of a smartphone, we can also look at the raw touch data of each "pixel" and the intensity it is detecting - resulting in the "blob" on the left. ML allows for the reconstruction of the "blob", *into the fingers location in 3D space*.



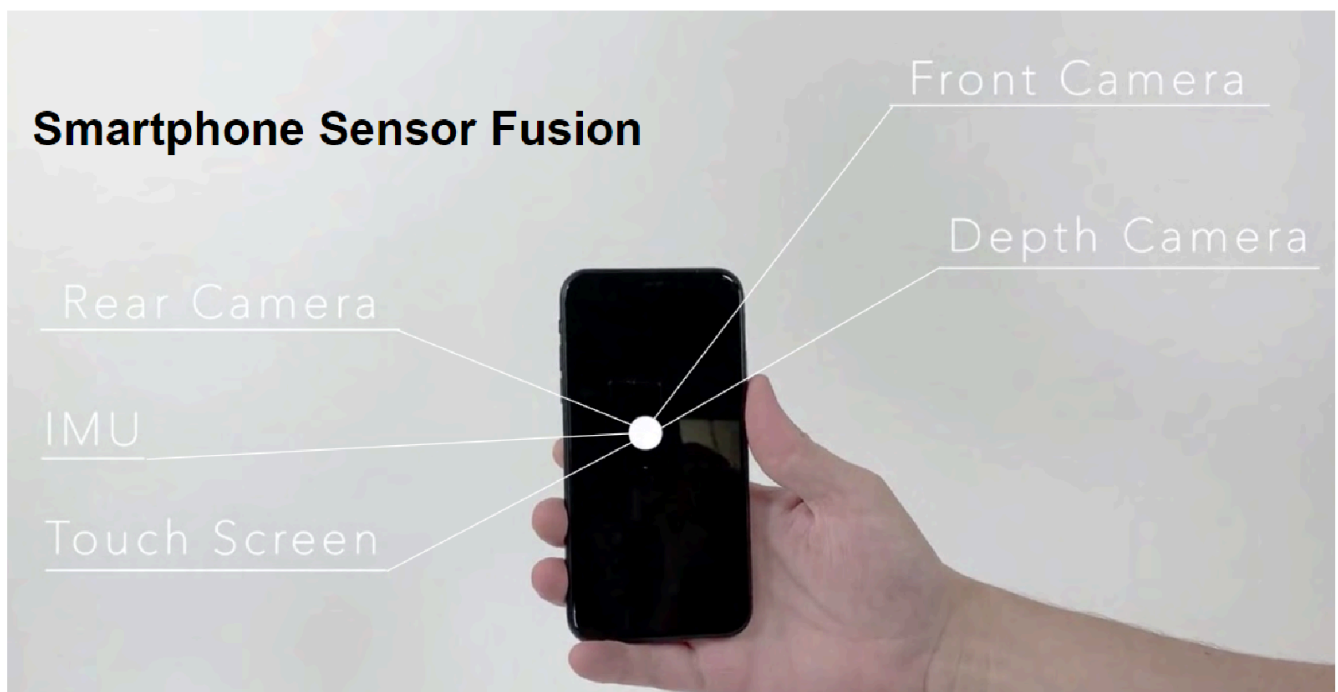
By analyzing this blob, ML models can extract features such as **pitch and yaw of the finger**; indicating the angle at which the finger touches the screen. This opens the door to more nuanced gestures and controls.

Moreover, ML can potentially identify **which finger is touching the screen**, enabling **multi-finger differentiation** for UI interactions. Like rotating the finger or tilting it to tilt a 3D-model if a car on screen.

🔗 Finger-Specific Interaction Paradigm

"A finger could draw in blue in a drawing app, while another finger is mapped to draw in red."

This kind of interaction significantly enhances expressiveness in touch-based interfaces.



It is even possible to just predict the pose of the user by taking all sensor readings of the phone in his hand!

❓ Why not use hand crafted algorithms?

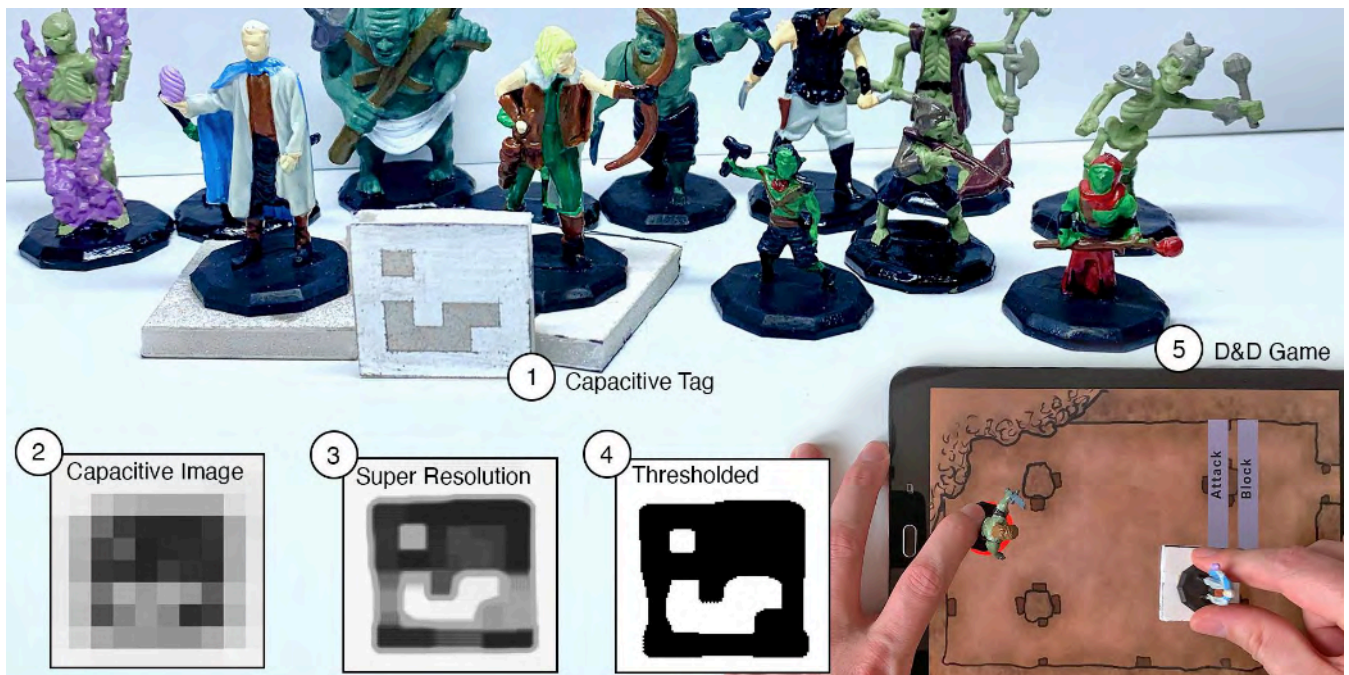
Because these often struggle with variability and complexity in real-world data. They can be brittle, meaning slight variations in input can lead to large errors, and they require significant human effort to design and maintain.

The shift from handcrafted algorithms to deep learning methods enables more robust and adaptable systems for human-computer interaction, as the models can automatically extract complex patterns and generalize better to new scenarios.

Super-Resolution for Marker Detection

Capacitive displays have limited spatial resolution. However, with ML, particularly **Super Resolution techniques**, we can artificially enhance the resolution of capacitive input.

This becomes especially useful when trying to detect **fiducial markers**, as physical tags placed on a screen that trigger specific UI responses.



A visual pattern (e.g., a moon or sun) recognized by a system to trigger an event or identify a location.

ML is can be used to simulate and recognize these markers more efficiently.

Adversarial Network (GAN) Simulations

Generative Adversarial Networks (GANs):

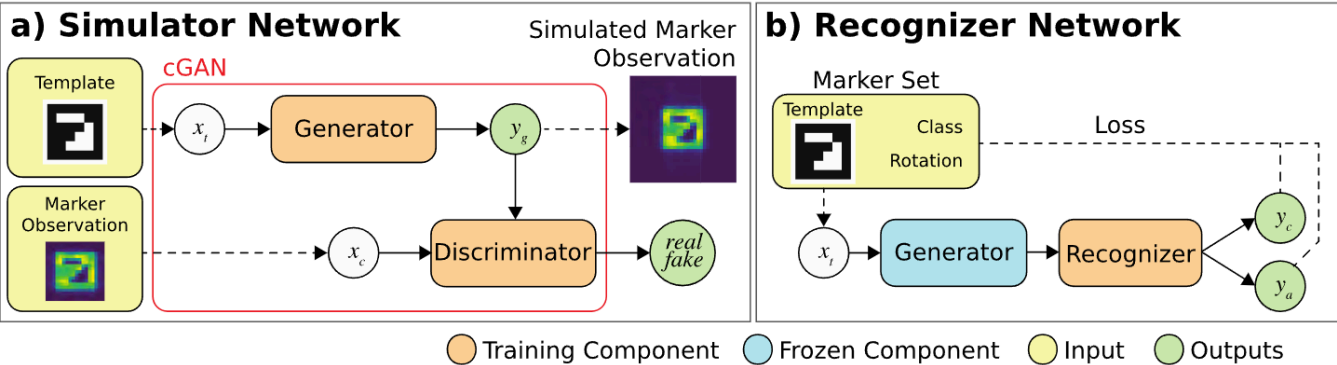
GANs consist of two neural networks: a **generator** and a **discriminator**, trained in opposition. The generator creates fake data, while the discriminator tries to distinguish real from fake. Over time, this adversarial training improves both.

Here GANs simulate how fiducial markers would look when pressed against a capacitive screen. Instead of capturing thousands of real examples for training, a **template + recording + simulation** pipeline is used:

Step	Description
Template Design	A designer creates a visual marker (e.g., a sun shape).
Single Recording	One real capacitive input of the marker is recorded.
GAN Simulation	The generator creates many variations based on the initial recording.

GANs don't just memorize

A common misconception is that GANs simply "augment" data like filters. In reality, they **learn to generate** new, plausible samples by understanding patterns in the input space. This makes them powerful tools for creating **synthetic but realistic** training data.

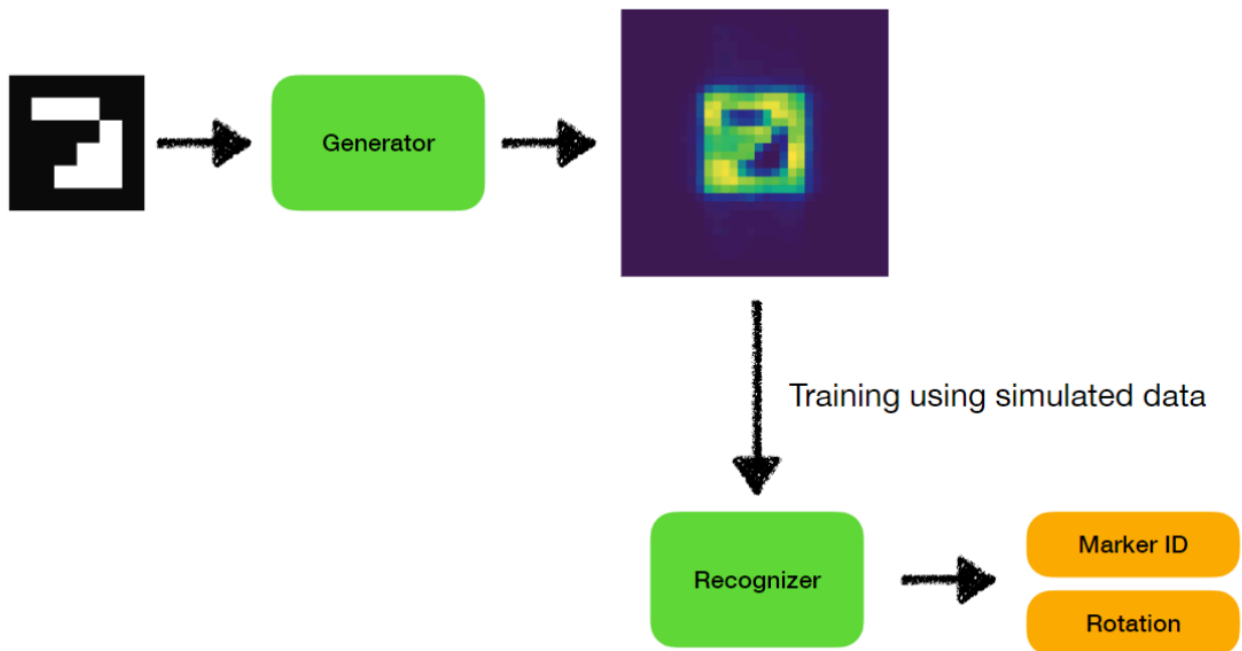
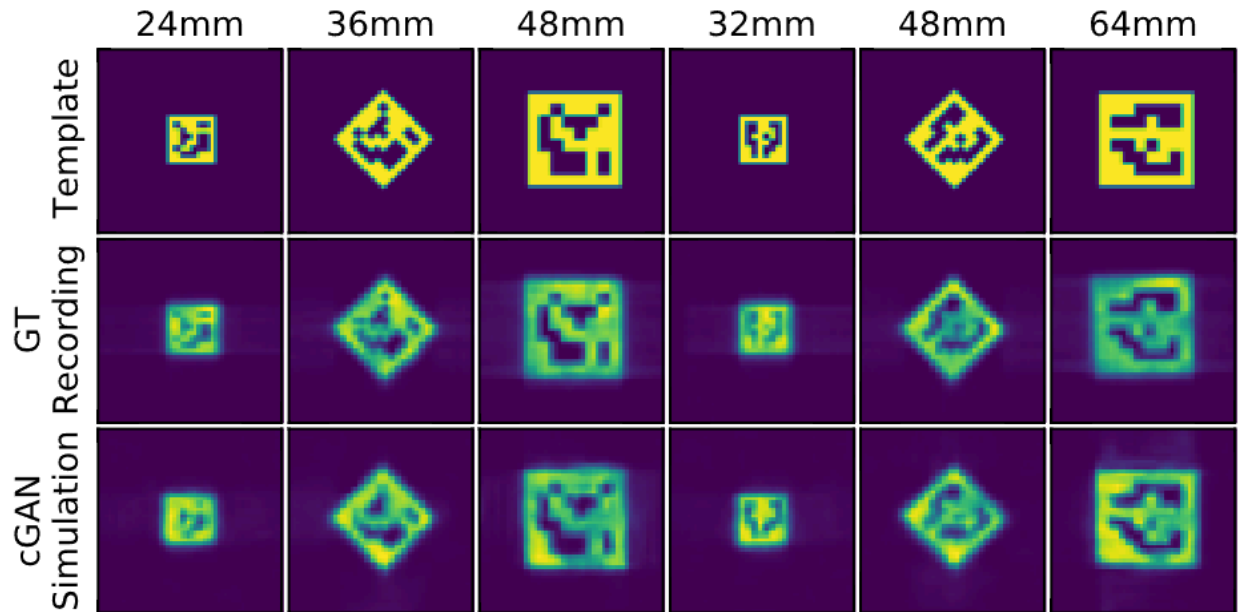


Simple Explanation of the concept:

More in detail explanation below

Feature	Simulator Network	Recognizer Network
Input	Fiducial Marker image	Simulated or real capacitive image
Output	Simulated capacitive image	Predicted label or object ID
Purpose	Forward model: <i>generate sensor response</i>	Inverse model: <i>infer object from sensor data</i>
Type	Generative (e.g. GAN or CNN decoder)	Discriminative (e.g. CNN classifier or encoder)
Use Case	Training data augmentation, sensor simulation	Object recognition, classification

General Adversarial Network Simulations



a) Simulator Network

The Simulator Network is designed to generate synthetic images of fiducial markers pressed against a capacitive screen, aiming to be indistinguishable from real observations. It operates as a Generative Adversarial Network (GAN).

- **Inputs:** The Simulator Network takes two primary inputs:
 - A **template** of the fiducial marker.
 - A **marker observation**, which could be a real or synthetic image representing how the marker would look on the screen.
- **Components:**
 - **Generator:** This component is responsible for creating new, synthetic images of the fiducial markers. Its goal is to generate images that can fool the Discriminator.
 - **Discriminator:** This component acts as a critic. It receives both real marker observations and the synthetic images generated by the Generator. Its task is to distinguish between real and fake (generated) images.

- **Process:** The Generator and Discriminator are trained in an adversarial manner. The Generator tries to produce images that the Discriminator classifies as "real," while the Discriminator tries to correctly identify "real" images as real and "fake" images as fake.
- **Output:** The Simulator Network ultimately aims to output a new, generated image that is highly realistic, and the Discriminator provides a probability score indicating whether an input image (real or generated) is perceived as "real" or "fake."

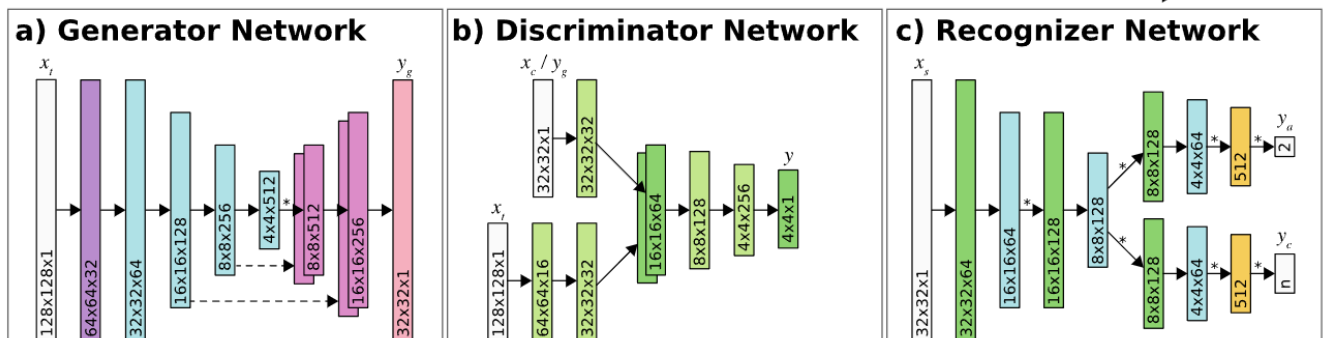
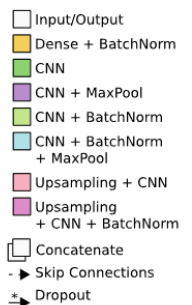
b) Recognizer Network

The Recognizer Network is designed to infer the class and rotation of fiducial markers from an input image. One key here is the frozen Generator, that tries to make the recognition more robust.

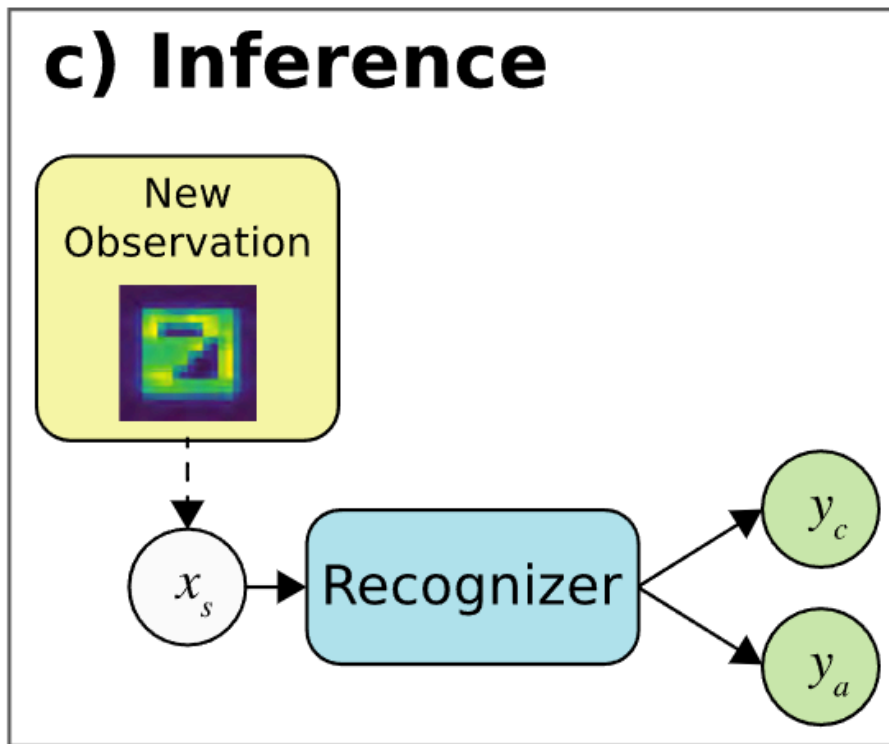
- **Input (x_1):** This is the raw input image containing a fiducial marker, which the network needs to analyze.
- **Generator:** Instead of being a separate training component as in a typical GAN, here the Generator is "frozen." This means its weights and parameters are fixed and **not updated** during the training of the Recognizer Network.
 - Purpose of **Frozen Generator:** It (likely pre-trained from the Simulator Network) is used to synthesize a **canonical or normalized representation** of the input marker. It might transform the input x_1 into a standardized version of the marker (e.g., removing noise, standardizing lighting, aligning, ...) before it reaches the recognizer. This pre-processing step helps the Recognizer focus on essential features for class and rotation prediction, making it more robust to variations in input.
- **Recognizer:** It processes the input image to identify the marker and its properties.
- **Loss:** During the training of the Recognizer Network, a loss function measures the discrepancy between the network's predicted outputs (y_a and y_c). The network's parameters (within the Recognizer component, as the Generator is frozen) are adjusted to minimize this loss.
- **Outputs:** The Recognizer Network produces two primary outputs:
 - y_a : Represents the **class** of the detected fiducial marker
 - y_c : Represents the **rotation**

These networks **decouple the design from the training**, meaning designers can create new markers without needing a new ML training cycle.

Internal GAN Structure



Deployment (no GAN)



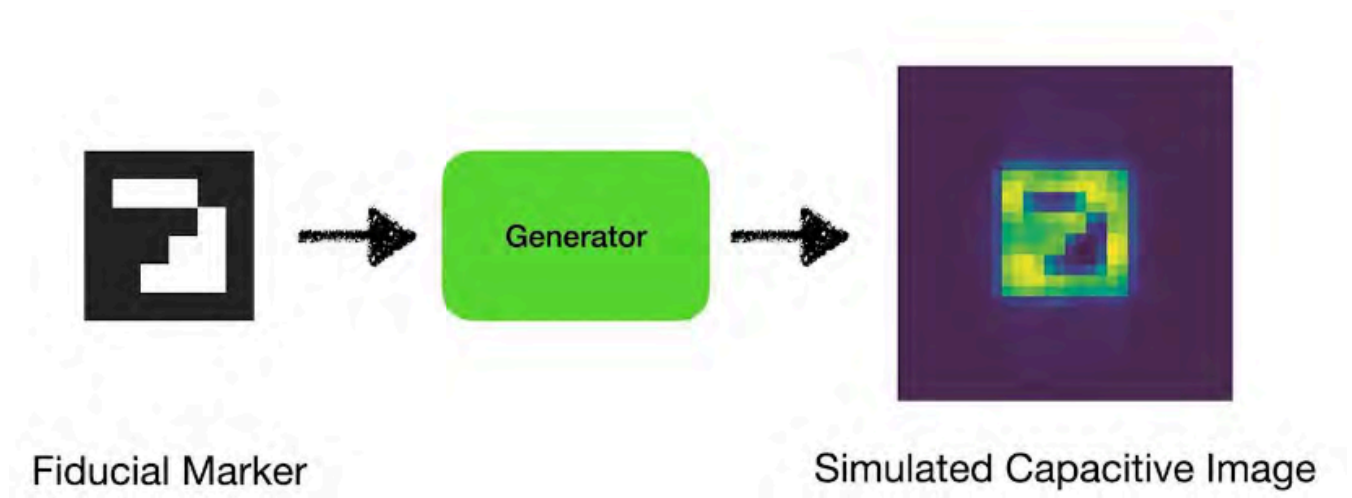
c) Inference Component

The **deployment phase** is where the trained Recognizer Network is used to make predictions on new, unseen data. Now it is not a GAN, it is basically just a pre-trained algorithm.

- **New Observation (x_s):** This is the raw input image of a fiducial marker, acquired.
- **Recognizer (Frozen):** The previously trained Recognizer Network is now used in an inferential capacity. It is "frozen," meaning its weights and parameters are fixed and **not updated** during this phase. It simply takes the input and make it's prediction!
 - It is "frozen" so it is consistent and efficient in real-world application, as training is computationally intensive and done offline.
 - To improve this design the input x_s could again first pass through the pre-processing step (like the "Frozen Generator"), but not here!
- **Outputs):**
 - y_a : The predicted **class**
 - y_c : The predicted **rotation**

🔗 One-Shot Learning

This is a technique where a model learns to recognize an object/class from a single example, ideal for scenarios with few available samples.



? *How does the GAN-based simulation approach improve marker recognition?*

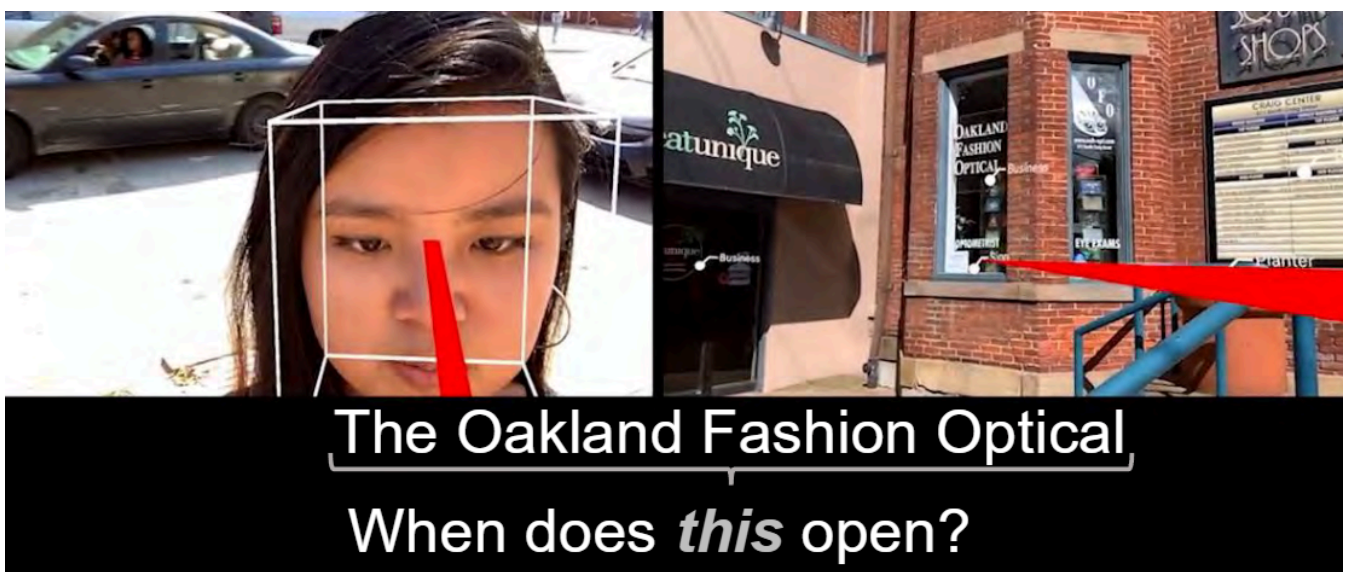
It removes the need for extensive real-world training data and allows for fast adaptation to new marker designs with minimal latency.

Context / Gaze Aware Voice Assistants

Context-aware assistants combine multiple sensor inputs to interpret what a user might be focusing on. A smartphone could:

- Use the **front-facing camera** to detect **head orientation** (gaze direction)
- Combine this with **rear-facing camera input** (scene view)
- Add **GPS data** and **environmental sensors**

This multimodal input helps the assistant infer the user's intention. E.g., when someone looks at a café, the assistant might offer its opening hours.

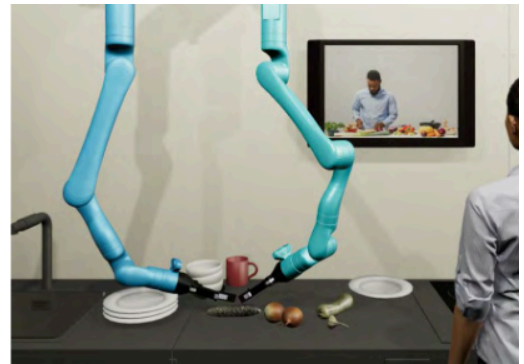


? *How can gaze data enhance voice assistant interaction?*

It allows the assistant to proactively understand what the user is referencing, enabling more seamless and natural interactions in the real world.

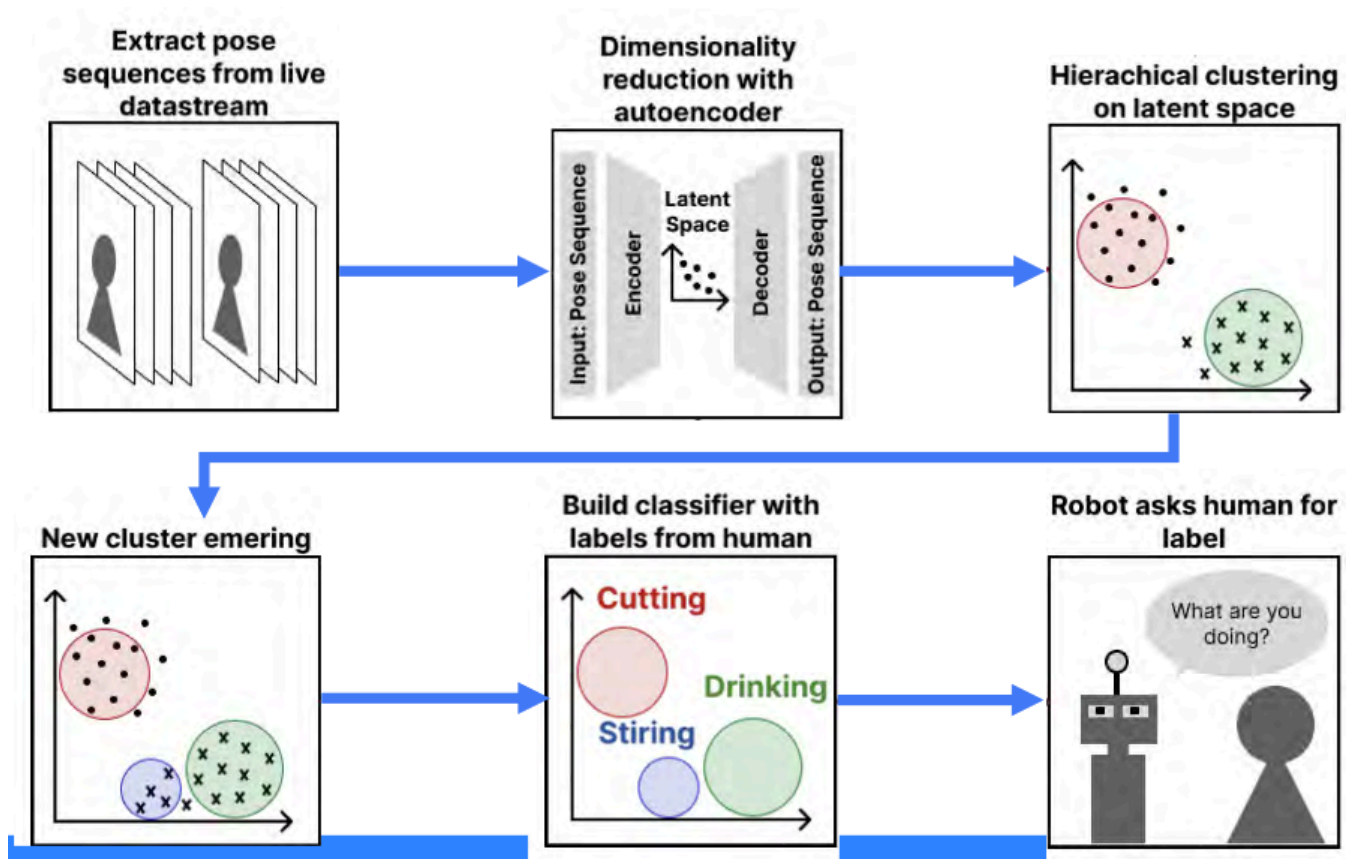
Dialog Systems and Human-Robot Collaboration

Dialog systems that operate in physical environments must **share control** with the user. This requires a negotiation process between human and machine.



Negotiation Process:

A method where the system and user align their goals through dialog. The system asks for clarifications, confirms understanding, and adapts its behavior accordingly.



These systems are often **Human-in-the-Loop**:

Human-in-the-Loop Systems:

Systems where humans provide feedback or make decisions during the process, improving the accuracy and safety of autonomous systems.

🔗 **What is the role of negotiation in human-in-the-loop dialog systems?**

It helps align machine behavior with human intent, especially in dynamic, shared environments where assumptions must be confirmed.

Models

We will look into some these further down the line

Models	“Deep” Learning Methods
- “Traditional” Machine Learning - Support Vector Machines - Decision Trees - Random Forest - ...	- Neuronal Networks - Convolutional Neuronal Networks - Recurrent Neural Network (RNN) - Generative Adversarial Network (GAN) - ...
